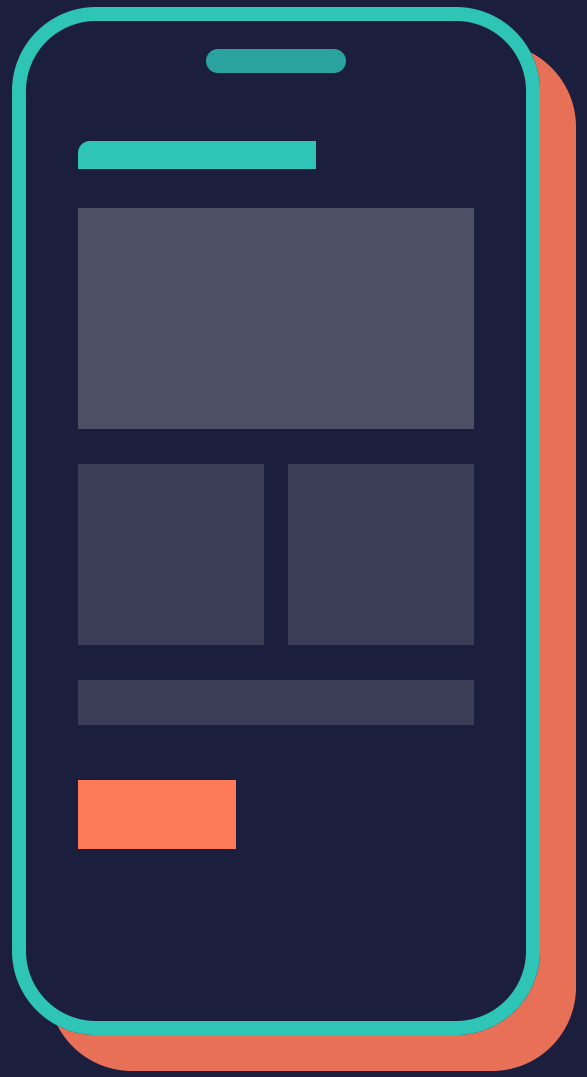
Navigation et cycle de vie

Cours 3 – Expo Router, données distantes et listes

Chris CHEVALIER



Sommaire

01 Naviguer sur mobile et sur le web

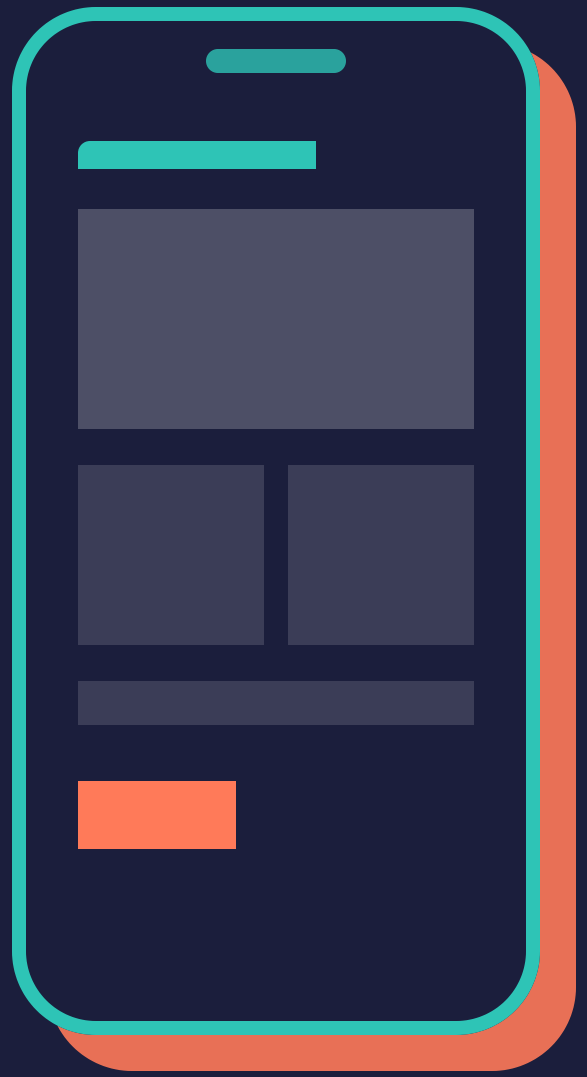
02 Expo Router : les fichiers sont les routes

03 Onglets, pile et routes dynamiques

04 Les données distantes

05 Le cycle de vie

06 Les listes volumineuses



01

Naviguer sur mobile et sur le web

Deux logiques à réconcilier



Deux façons de naviguer

	Sur le web	Sur mobile
Ce qui définit l'écran	L'URL dans la barre d'adresse	Une **pile** d'écrans en mémoire
Le retour	Le bouton du navigateur	La flèche de l'en-tête, le geste ou le bouton système
Les sections	Un menu	Des **onglets** en bas de l'écran
Ouvrir un contenu de l'extérieur	Un lien partagé	Un **lien profond** (deep link) qui ouvre l'application sur le bon écran

Avec **Expo Router**, chaque écran a une URL. La même navigation produit une pile sur mobile et des adresses dans le navigateur.

La pile d'écrans

1. Onglet « Séjours »

`/`

Le catalogue est la base de la pile

2. On empile

`/stays/s3`

Le détail se pose par-dessus, avec une flèche de retour

3. On dépile

`/`

Le catalogue réapparaît **dans l'état où on l'avait laissé** (filtres, défilement)

> Expo Router s'appuie sur React Navigation, la bibliothèque de navigation de référence. Vous n'avez pas à la configurer vous-mêmes.

02

Expo Router : les fichiers sont les routes

Le routage par fichiers



Un fichier, une URL

src/app/

```
src/app/
├── _layout.tsx          pile racine
├── +not-found.tsx       URL inconnue
├── (tabs)/
│   ├── _layout.tsx      onglets
│   ├── index.tsx        /
│   └── favorites.tsx    /favorites
└── stays/
    └── [id].tsx         /stays/s1, /stays/s2…
```

- `_layout.tsx` : le **navigateur** (pile ou onglets) de son dossier
- `(tabs)` : un **groupe**. Les parenthèses n'apparaissent pas dans l'URL
- `[id]` : un **segment dynamique**, récupéré comme paramètre
- `+not-found` : l'écran des adresses inconnues

La pile racine

src/app/_layout.tsx

```tsx
import { Stack } from "expo-router";

import { FavoritesProvider } from "@/context/favorites-context";

// Ouvert par son URL, un séjour garde un retour vers les onglets
export const unstable_settings = { anchor: "(tabs)" };

export default function RootLayout() {
  return (
    <FavoritesProvider>
      <Stack>
        <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
        <Stack.Screen name="stays/[id]" options={{ title: "Séjour" }} />
      </Stack>
    </FavoritesProvider>
  );
}
```

- Les onglets forment **un seul écran** de la pile, sans en-tête (ils ont le leur)
- Le détail se pose par-dessus
- `anchor` : sans lui, une page ouverte par son URL n'a pas de flèche de retour

03

Onglets, pile et routes dynamiques

Construire le parcours de l'utilisateur



Les onglets

src/app/(tabs)/_layout.tsx

```tsx
import Ionicons from "@expo/vector-icons/Ionicons";
import { Tabs } from "expo-router/js-tabs";
const ACTIVE = "#0B7A70";
const INACTIVE = "#8C90B4";

export default function TabsLayout() {
  return (
    <Tabs screenOptions={{
      tabBarActiveTintColor: ACTIVE, tabBarInactiveTintColor: INACTIVE }}>
      <Tabs.Screen name="index" options={{
        title: "Séjours",
        tabBarIcon: ({ focused, size }) => (
          <Ionicons name="bed-outline" size={size}
            color={focused ? ACTIVE : INACTIVE} />
        ),
      }} />
      <Tabs.Screen name="favorites" options={{ title: "Favoris" /* … */ }} />
    </Tabs>
  );
}
```

- Depuis le SDK 58, importez `Tabs` depuis `expo-router/js-tabs`. L'import depuis `expo-router` est **déprécié**
- `name` reprend le nom du fichier
- Icônes : `@expo/vector-icons`, à installer avec `npx expo install`

Naviguer : `Link` et `useRouter`

Un lien (dans favorites.tsx)

```tsx
<Link href={{
  pathname: "/stays/[id]",
  params: { id: item.id },
}}>
  {item.title}
</Link>
```

Par le code (dans index.tsx)

```tsx
const router = useRouter();
const openStay = (id: string) =>
  router.push({ pathname: "/stays/[id]", params: { id } });
```

Méthode	Effet
`push(href)`	Empile un écran
`back()`	Revient à l'écran précédent
`replace(href)`	Remplace l'écran actuel (après une connexion, par exemple)
`navigate(href)`	Va à l'écran, sans l'empiler s'il existe déjà

Les **routes typées** (`typedRoutes` dans `app.json`) font signaler par TypeScript un lien vers une page inexistante.

Une route dynamique

src/app/stays/[id].tsx (extrait)

```tsx
export default function StayScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const state = useStay(id);

  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "not-found")
    return <StateMessage message="Ce séjour est introuvable." />;
  return <StayDetails stay={state.stay} />;
}

function StayDetails({ stay }: { stay: Stay }) {
  const weather = useWeather(stay.latitude, stay.longitude);
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Stack.Screen options={{ title: stay.title }} />
      {/* photo, titre, prix… */}
      <WeatherBadge state={weather} />
    </ScrollView>
  );
}
```

- `useLocalSearchParams` lit `id` dans l'URL
- L'écran ne reçoit **que l'identifiant**, jamais l'objet entier : il recharge ses données. C'est ce qui permet d'ouvrir la page depuis un lien
- `<Stack.Screen options>` change le titre de l'en-tête
- Un identifiant inconnu est un cas prévu

URL et liens profonds

Sur le web

- L'URL `/stays/s3` s'affiche dans la barre d'adresse
- On peut la copier, la partager, la recharger : le même écran s'ouvre
- Le bouton retour du navigateur dépile

Sur mobile

- Le `scheme` d' `app.json` définit le préfixe : `cours3://stays/s3`
- Dans Expo Go, l'adresse devient `exp://<IP>:8081/--/stays/s3`
- Une notification ou un e-mail peut ouvrir l'application sur le bon séjour

Un paramètre d'URL vient de l'extérieur : **ne lui faites pas confiance**. Vérifiez qu'il existe avant de l'utiliser, comme le cas `not-found` du détail.

Quiz

Quelle URL correspond au fichier `src/app/(tabs)/favorites.tsx` ?

> `/favorites` : les groupes entre parenthèses n'apparaissent pas dans l'URL.

Pourquoi passer l'identifiant du séjour au détail plutôt que l'objet complet ?

> L'URL doit suffire à afficher la page : ouverte depuis un lien ou rechargée, elle n'a que l'identifiant. Le détail recharge donc ses données.

04

Les données distantes

Consommer une API publique



Un service qui appelle une API

src/services/weather-service.ts (extrait)

```ts
// Forme de la réponse de l'API, limitée à ce qu'on utilise
type OpenMeteoResponse = {
  current: { temperature_2m: number; wind_speed_10m: number; weather_code: number };
};
export async function fetchWeather(
  latitude: number, longitude: number
): Promise<Weather> {
  const url =
    "https://api.open-meteo.com/v1/forecast" +
    `?latitude=${latitude}&longitude=${longitude}` +
    "&current=temperature_2m,wind_speed_10m,weather_code";
  const response = await fetch(url);
  if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`);
  const data: OpenMeteoResponse = await response.json();
  // On convertit la réponse de l'API en objet du modèle
  return {
    temperature: data.current.temperature_2m,
    windSpeed: data.current.wind_speed_10m,
    code: data.current.weather_code,
  };
}
```

- `fetch` fonctionne de la même façon sur le web et sur mobile
- `fetch` ne lève **pas** d'erreur sur une réponse 404 ou 500 : testez `response.ok`
- Le service **traduit** la réponse de l'API dans le vocabulaire du modèle. Si l'API change, seul ce fichier change

Le contrôleur et la vue

src/hooks/use-weather.ts (extrait)

```ts
export type WeatherState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; weather: Weather };

export function useWeather(
  latitude: number, longitude: number
) {
  const [state, setState] =
    useState<WeatherState>({ status: "loading" });
  // … useEffect qui appelle fetchWeather()
  return state;
}
```

src/components/weather-badge.tsx (extrait)

```tsx
export function WeatherBadge({ state }: WeatherBadgeProps) {
  return (
    <View style={styles.badge}>
      {state.status === "loading" &&
        <ActivityIndicator />}
      {state.status === "error" &&
        <Text>{state.message}</Text>}
      {state.status === "success" && (
        <Text style={styles.text}>
          {describeWeather(state.weather.code)} ·{" "}
          {Math.round(state.weather.temperature)} °C
        </Text>
      )}
    </View>
  );
}
```

> Même structure qu'au cours 2 : le service parle à l'API, le hook tient l'état, la vue affiche ce qu'on lui donne.

Ce qui peut mal se passer

Situation	Ce qui se passe	Ce que voit l'utilisateur
Pas de réseau	`fetch` lève une erreur	Un message et « Réessayer »
Erreur 404 ou 500	`response.ok` vaut `false`	Un message et « Réessayer »
Réponse lente	La promesse attend	Un indicateur de chargement
L'écran est quitté avant la réponse	Le drapeau `cancelled` ignore le résultat	Rien

> Une clé d'API placée dans le code de l'application est **lisible par tous** : le code JavaScript est livré avec l'application. N'y mettez jamais de secret. Préférez des API sans clé ou passez par votre propre serveur.

05

Le cycle de vie

Écrans et application



Un écran ne se démonte pas toujours

src/app/(tabs)/favorites.tsx (extrait)

```tsx
export default function FavoritesScreen() {
  const { state, reload } = useStays();
  const { isFavorite } = useFavorites();

  // À chaque fois que l'onglet redevient visible, on recharge les séjours
  useFocusEffect(
    useCallback(() => {
      reload();
    }, [reload])
  );
  // …
}
```

- Un onglet quitté reste **monté** : son `useEffect(…, [])` ne s'exécutera plus
- `useFocusEffect` (d'`expo-router`) s'exécute à **chaque** affichage de l'écran
- La fonction renvoyée, si vous en écrivez une, s'exécute quand l'écran perd le focus
- `useCallback` évite de relancer l'effet à chaque rendu

L'application passe en arrière-plan

src/hooks/use-is-app-active.ts

```ts
import { useEffect, useState } from "react";
import { AppState } from "react-native";

// true au premier plan, false en arrière-plan
export function useIsAppActive(): boolean {
  const [isActive, setIsActive] = useState(true);

  useEffect(() => {
    const subscription = AppState.addEventListener(
      "change",
      (status) => setIsActive(status === "active")
    );
    return () => subscription.remove();
  }, []);

  return isActive;
}
```

- `AppState` signale le premier plan (`"active"`) et l'arrière-plan (`"background"`)
- Sur le web, il suit la visibilité de l'onglet du navigateur
- Toujours **retirer l'écouteur** au démontage, sinon il s'accumule
- `useWeather` s'en sert pour recharger la météo au retour dans l'application

Quel outil pour quel moment ?

Moment	Outil	Exemple
Le composant apparaît pour la première fois	`useEffect(…, [])`	Charger le détail d'un séjour
Un paramètre change	`useEffect(…, [id])`	Recharger quand l'identifiant change
L'écran redevient visible	`useFocusEffect`	Rafraîchir l'onglet des favoris
L'application revient au premier plan	`AppState`	Recharger la météo
Le composant disparaît	la fonction renvoyée par l'effet	Retirer un écouteur, ignorer une réponse tardive

06

Les listes volumineuses

FlatList et SectionList



Une `FlatList` qui reste fluide

Prop ou pratique	Effet
`keyExtractor`	Une clé stable par élément : React réutilise les lignes au lieu de les recréer
`initialNumToRender`	Le nombre d'éléments affichés au premier rendu (10 par défaut)
`windowSize`	Combien d'écrans de contenu sont gardés en mémoire autour de la zone visible
`getItemLayout`	Si toutes les lignes ont la même hauteur, la liste n'a plus à les mesurer
Des éléments légers	Peu de vues imbriquées, des images à la bonne taille
Pas de calcul dans `renderItem`	Filtrer ou trier **avant**, dans le hook

Pour des milliers de lignes, FlashList (Shopify) réutilise encore mieux les éléments.

`SectionList` : une liste par groupes

src/models/stay.ts (extrait)

```ts
export type CitySection = { title: string; data: Stay[] };

// Regroupe des séjours par ville,
// au format attendu par SectionList
export function groupByCity(
  stays: Stay[]
): CitySection[] {
  const cities =
    [...new Set(stays.map((stay) => stay.city))].sort();
  return cities.map((city) => ({
    title: city,
    data: stays.filter((stay) => stay.city === city),
  }));
}
```

src/app/(tabs)/favorites.tsx (extrait)

```tsx
<SectionList
  sections={sections}
  keyExtractor={(stay) => stay.id}
  renderSectionHeader={({ section }) => (
    <Text style={styles.city}>{section.title}</Text>
  )}
  renderItem={({ item }) => (
    <Link href={{
      pathname: "/stays/[id]",
      params: { id: item.id },
    }}>
      {item.title}
    </Link>
  )}
/>
```

Chaque section a un `title` et un tableau `data`. Le regroupement est une **règle du modèle**, pure et testable.

Quiz

L'onglet « Favoris » charge ses données dans un `useEffect(…, [])`. J'ajoute un favori depuis le catalogue puis je reviens. Que se passe-t-il ?

> L'onglet n'est pas rechargé : il est resté monté, l'effet ne se relance pas. Il faut `useFocusEffect`.

`fetch` reçoit une réponse 404. Le bloc `catch` s'exécute-t-il ?

> Non : `fetch` ne lève une erreur qu'en cas d'échec réseau. Il faut tester `response.ok` et lever l'erreur soi-même.

À retenir

- Avec Expo Router, **chaque fichier de `src/app/` est une route** et chaque écran a une URL
- `_layout.tsx` choisit le navigateur : pile (`Stack`) ou onglets (`Tabs` depuis `expo-router/js-tabs`)
- Une route dynamique reçoit un identifiant et **recharge** ses données
- Un service appelle l'API avec `fetch`, teste `response.ok` et traduit la réponse en modèle
- `useFocusEffect` pour un écran qui redevient visible, `AppState` pour l'application
- Des listes légères, filtrées en amont. `SectionList` pour des groupes

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

